# Основные категории SQL-команд

Разберем основные категории SQL-команд — с расшифровкой, назначением и примерами.

## DDL (Data Definition Language) — язык определения данных

**Назначение:** управление структурой базы данных («скелет» БД): создание, изменение и удаление объектов.

**Ключевые команды:**

- `CREATE` — создание новых объектов (таблиц, индексов, представлений, схем);
- `ALTER` — изменение существующих объектов (добавление/удаление столбцов, изменение типов данных);
- `DROP` — удаление объектов (таблицы, индексы и т.д.);
- `TRUNCATE` — очистка всех данных из таблицы без удаления её структуры;
- `RENAME` — переименование объектов;
- `COMMENT` — добавление комментариев к объектам.

**Примеры:**

```sql
-- Создание таблицы
CREATE TABLE employees (
    id INT PRIMARY KEY,
    name VARCHAR(100)
);

-- Добавление столбца
ALTER TABLE employees ADD email VARCHAR(100);

-- Удаление таблицы
DROP TABLE employees;

-- Очистка данных таблицы
TRUNCATE TABLE employees;
```

**Особенности:**

- автоматически выполняют неявный `COMMIT` (изменения нельзя отменить);
- часто требуют повышенных привилегий;
- при `DROP` данные теряются безвозвратно.

---

## DML (Data Manipulation Language) — язык манипулирования данными

**Назначение:** работа с содержимым таблиц: добавление, изменение, удаление и извлечение данных.

**Ключевые команды:**

- `SELECT` — извлечение данных из таблиц;
- `INSERT` — добавление новых записей;
- `UPDATE` — изменение существующих записей;
- `DELETE` — удаление записей по условию;
- `MERGE` — комбинированная операция вставки/обновления (есть не во всех СУБД).

**Примеры:**

```sql
-- Извлечение данных
SELECT * FROM employees WHERE department = 'IT';
```

```sql
-- Добавление записи
INSERT INTO employees (id, name) VALUES (1, 'Анна Иванова');

-- Обновление данных
UPDATE employees SET salary = 50000 WHERE id = 1;

-- Удаление записей
DELETE FROM employees WHERE status = 'уволен';
```

**Особенности:**

- операции можно отменить через `ROLLBACK` до `COMMIT`;
- `SELECT` не изменяет данные;
- `INSERT`, `UPDATE`, `DELETE` могут запускать триггеры.

---

## DQL (Data Query Language) — язык запросов

**Назначение:** только чтение данных (часто считают частью DML, но выделяют отдельно из-за специфики).

**Ключевая команда:** * `SELECT` — выборка данных с фильтрацией, сортировкой, группировкой.

**Пример:**

```sql
SELECT name, department
FROM employees
WHERE salary > 40000
ORDER BY name;
```

---

## DCL (Data Control Language) — язык управления доступом

**Назначение:** контроль прав пользователей и ролей для обеспечения безопасности данных.

**Ключевые команды:**

- `GRANT` — предоставление прав доступа;
- `REVOKE` — отзыв ранее предоставленных прав;
- `DENY` — явный запрет действий (в некоторых СУБД, например, Microsoft SQL Server).

**Примеры:**

```sql
-- Предоставление прав
GRANT SELECT, INSERT ON employees TO hr_manager;

-- Отзыв прав
REVOKE DELETE ON employees FROM junior_developer;
```

---

## TCL (Transaction Control Language) — язык управления транзакциями

**Назначение:** обеспечение целостности данных при выполнении группы операций («всё или ничего»).

**Ключевые команды:**

- COMMIT — фиксация изменений (делает их постоянными);
- ROLLBACK — отмена изменений до последней точки сохранения;
- SAVEPOINT — установка промежуточной точки сохранения внутри транзакции;
- BEGIN / START TRANSACTION — начало транзакции.

**Пример:**

```sql
BEGIN TRANSACTION;
UPDATE accounts SET balance = balance - 1000 WHERE user_id = 1;
UPDATE accounts SET balance = balance + 1000 WHERE user_id = 2;
COMMIT; -- или ROLLBACK при ошибке
```

---

### Краткий итог: сравнение категорий

| Категория | Назначение | Основные команды | Изменяет данные? | Можно отменить? |
|---|---|---|---|---|
| **DDL** | Структура БД | CREATE, ALTER, DROP, TRUNCATE | Нет (только структуру) | Нет |
| **DML** | Содержимое таблиц | INSERT, UPDATE, DELETE, MERGE | Да | Да (до COMMIT) |
| **DQL** | Чтение данных | SELECT | Нет | — |
| **DCL** | Права доступа | GRANT, REVOKE, DENY | Нет (метаданные) | Да |
| **TCL** | Целостность операций | COMMIT, ROLLBACK, SAVEPOINT | Да (группой) | Да (ROLLBACK) |

Понимание этих категорий помогает:

- структурировать код;
- выбирать правильные команды для задач;
- обеспечивать безопасность и целостность данных;
- готовиться к собеседованиям и сертификациям.

## Порядок выполнения SQL-запроса (логический)

Важно различать **порядок написания** SQL-запроса и **порядок его выполнения** — они не совпадают. Разберу пошагово логический порядок обработки команд.

### Порядок выполнения

1. **FROM / JOIN**
   - Определяются источники данных: таблицы и соединения (JOIN).
   - Формируется базовый набор строк для дальнейшей обработки.
2. **WHERE**
   - Фильтрация строк: отбрасываются записи, не удовлетворяющие условиям.
   - Обработка происходит **до группировки**.
3. **GROUP BY**
   - Данные группируются по указанным столбцам.
   - Каждая группа содержит строки с одинаковыми значениями в указанных столбцах.
4. **Агрегирующие функции** (SUM, COUNT, AVG и т.д.)
   - Вычисляются над данными в группах.
   - Например, COUNT(*) считает количество строк в каждой группе.
5. **HAVING**

- Фильтруются группы, не соответствующие условиям.
- В отличие от `WHERE`, работает с **результатами агрегации**.
- Пример: `HAVING COUNT(*) > 5` оставит только группы с более чем 5 строками.

6. **Оконные функции**
   - Выполняются вычисления по «окнам» строк (определённым `PARTITION BY` и `ORDER BY`).
   - Примеры: `ROW_NUMBER()`, `LAG()`, `SUM() OVER()`.
7. **`SELECT`**
   - Выбираются столбцы и выражения для итогового результата.
   - Создаются алиасы (псевдонимы) через `AS`.
   - На этом этапе становятся доступны для использования результаты оконных функций.
8. **`DISTINCT`**
   - Удаляются дублирующиеся строки из результата.
   - Применяется после выбора столбцов.
9. **`UNION` / `INTERSECT` / `EXCEPT`**
   - Объединяются результаты нескольких запросов.
   - `UNION` — объединение, `INTERSECT` — пересечение, `EXCEPT` — разность.
10. **`ORDER BY`**
    - Сортировка строк по указанным столбцам или выражениям.
    - Можно использовать алиасы из `SELECT`.
    - По умолчанию — по возрастанию (`ASC`), для убывания — `DESC`.
11. **`OFFSET`**
    - Пропускаются указанное количество строк.
    - Часто используется для постраничной навигации.
    - Пример: `OFFSET 10` пропустит первые 10 строк.
12. **`LIMIT` / `FETCH` / `TOP`**
    - Ограничивается количество возвращаемых строк.
    - `LIMIT 5` вернёт не более 5 строк.
    - Выполняется **в самом конце**, после всех остальных операций.

---

## Важные нюансы

- **Оптимизатор СУБД** может менять реальный порядок выполнения для повышения эффективности, но **логический порядок** остаётся таким, как описано выше.
- Нельзя использовать алиас из `SELECT` в `WHERE` или `GROUP BY` — на момент их выполнения `SELECT` ещё не выполнен.
- Оконные функции вычисляются **после** агрегации и `HAVING`, поэтому их нельзя использовать в `WHERE`.
- Для обхода ограничений можно использовать **CTE** (`WITH`) или подзапросы: сначала вычислить нужные значения, затем фильтровать по ним.

---

## Пример

Запрос:

```sql
SELECT name, SUM(amount) AS total
FROM orders
JOIN customers ON orders.customer_id = customers.id
WHERE order_date >= '2023-01-01'
GROUP BY name
HAVING SUM(amount) > 1000
ORDER BY total DESC
LIMIT 10;
```

Порядок выполнения: 1. Соединяем таблицы `orders` и `customers` (`FROM + JOIN`). 2. Оставляем только заказы с 2023 года (`WHERE`). 3. Группируем по имени клиента (`GROUP BY`). 4. Считаем сумму заказов для каждого клиента (`SUM(amount)`). 5. Отбрасываем клиентов с суммой меньше 1000 (`HAVING`). 6. Выбираем столбцы `name` и `total` (`SELECT`). 7. Сортируем по убыванию суммы (`ORDER BY`). 8. Оставляем топ-10 клиентов (`LIMIT`).

# О базе данных Chinook

**База данных примеров Chinook** — это общедоступная база данных с открытым исходным кодом, созданная разработчиком Microsoft Линном Рутом.

Она широко используется в обучающих материалах и курсах.

В каждой таблице базы данных хранится достоверная информация о продажах музыки: - **album** — названия альбомов и исполнители - **artist** — информация об исполнителях - **track** — отдельные музыкальные треки с указанием альбома, жанра и типа носителя - **genre** — список доступных музыкальных жанров - **customer** — контактные данные и местоположение клиентов - **invoice** — счета клиентов - **invoice_item** — позиции, включенные в каждый счет - **playlist** — плейлисты, созданные пользователями - **playlist_track** — сопоставление плейлистов и треков - **employee** — данные о сотрудниках и представителях службы поддержки

# Схема базы данных Chinook

# Примеры запросов при работе с базой данных Chinook

## Подключаем необходимые модули

```
import sqlite3
from pathlib import Path

import matplotlib.pyplot as plt
import pandas as pd
from IPython.display import Markdown
```

```
db_path = Path(Path.cwd().parent) / "db/chinook.db"
conn = sqlite3.connect(db_path)
```

## Выбрать все записи из таблицы `customer`

Запрос `SELECT * FROM customer LIMIT 5;` извлекает все столбцы из таблицы `customer` и возвращает первые 5 строк данных — остальные записи игнорируются.

```
sql = "SELECT FirstName, LastName, Company, Address  FROM customer LIMIT 5;"
data = pd.read_sql_query(sql, conn)
display(Markdown(data.to_markdown(index=False)))
```

| FirstName | LastName | Company | Address |
|---|---|---|---|
| Luís | Gonçalves | Embraer - Empresa Brasileira de Aeronáutica S.A. | Av. Brigadeiro Faria Lima, 2170 |
| Leonie | Köhler | nan | Theodor-Heuss-Straße 34 |
| François | Tremblay | nan | 1498 rue Bélanger |
| Bjørn | Hansen | nan | Ullevålsveien 14 |

| FirstName | LastName | Company | Address |
|---|---|---|---|
| František | Wichterlová | JetBrains s.r.o. | Klanova 9/506 |

## Выбрать все записи из таблицы customer по определенным столбцам

Запрос `SELECT FirstName, LastName, Country FROM customer LIMIT 5;` извлекает из таблицы `customer` только три указанных столбца (`FirstName`, `LastName` и `Country`) и возвращает первые 5 строк данных — все остальные строки игнорируются благодаря ограничению `LIMIT 5`.

```
sql = "SELECT FirstName, LastName, Country FROM customer LIMIT 5;"
data = pd.read_sql_query(sql, conn)
display(Markdown(data.to_markdown(index=False)))
```

| FirstName | LastName | Country |
|---|---|---|
| Luís | Gonçalves | Brazil |
| Leonie | Köhler | Germany |
| François | Tremblay | Canada |
| Bjørn | Hansen | Norway |
| František | Wichterlová | Czech Republic |

## Посчитать кол-во клиентов каждой стране

Запрос `SELECT Country, COUNT(*) AS customer_count FROM customer GROUP BY Country ORDER BY customer_count DESC;` группирует записи из таблицы `customer` по столбцу `Country`, подсчитывает количество клиентов в каждой стране (`COUNT(*)` и присваивает результату псевдоним `customer_count`), а затем сортирует полученные данные по убыванию числа клиентов — так, что страна с наибольшим количеством клиентов оказывается первой в итоговом списке.

```
sql = """SELECT Country, Count(*) AS customer_count
    FROM customer
    GROUP BY Country
    ORDER BY customer_count DESC;"""
data = pd.read_sql_query(sql, conn)
display(Markdown(data.head(5).to_markdown(index=False)))
```

| Country | customer_count |
|---|---|
| USA | 13 |
| Canada | 8 |
| France | 5 |
| Brazil | 5 |
| Germany | 4 |

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
ax1.plot(data["Country"].head(5), data["customer_count"].head(5), label="Количество клиент
ax1.set_ylabel("Кол-во клиентов")
ax1.set_xlabel("Страны")
ax1.set_title("Top 5 стран по кол-ву клиентов")
ax1.legend()
ax1.grid(True, alpha=0.3)

ax2.plot(data["Country"].tail(5), data["customer_count"].tail(5), label="Количество клиент
```

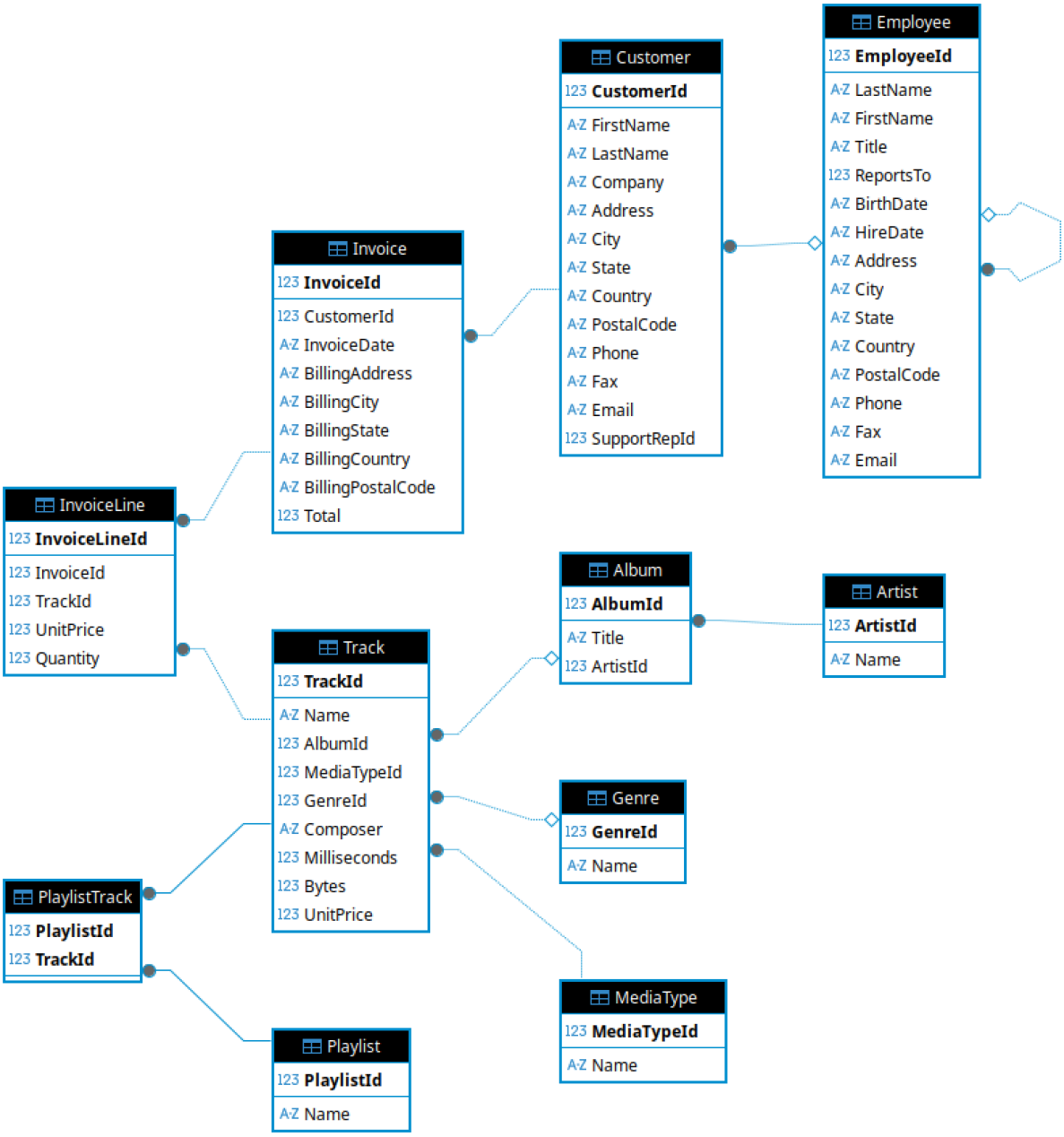


Figure 1: img_1

```
ax2.set_ylabel("Кол-во клиентов")
ax2.set_xlabel("Страны")
ax2.set_title("Top 5 стран по кол-ву клиентов")
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.show()
```

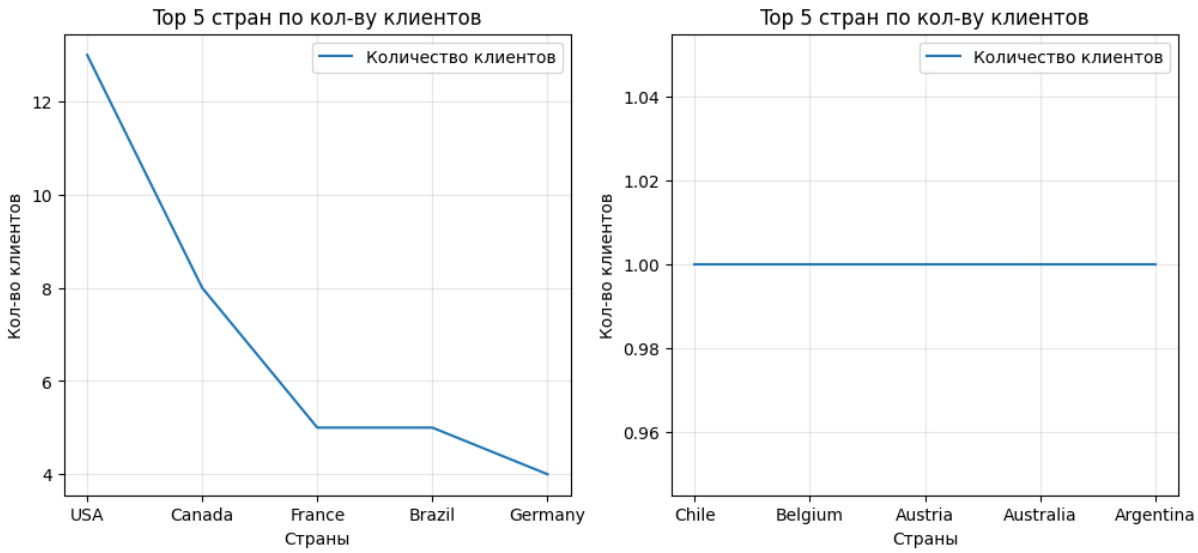


Figure 2: png

## Отчёт по продажам с разбивкой по клиентам и ответственным сотрудникам

Запрос объединяет данные из таблиц `customer`, `employee` и `invoice`, формирует полные имена клиента (`CustomerName`) и ответственного сотрудника (`EmployeeName`), подсчитывает количество счетов-фактур (`invoices`) и суммирует их общую стоимость (`total`) для каждого клиента. Результаты группируются по идентификатору клиента и данным сотрудника, а затем сортируются по убыванию общей суммы (`total`).

```
sql = """SELECT
    c.FirstName || ' ' || c.LastName AS CustomerName,
    e.FirstName || ' ' || e.LastName AS EmployeeName,
    COUNT(i.InvoiceId) AS invoices,
    SUM(i.Total) AS total
FROM customer c
LEFT JOIN employee e ON c.SupportRepId = e.EmployeeId
LEFT JOIN invoice i ON i.CustomerId  = c.CustomerId
GROUP BY c.CustomerId, e.FirstName, e.LastName
ORDER BY total DESC;"""
data = pd.read_sql_query(sql, conn)
display(Markdown(data.head(5).to_markdown(index=False)))
```

| CustomerName | EmployeeName | invoices | total |
|---|---|---|---|
| Helena Holý | Steve Johnson | 7 | 49.62 |
| Richard Cunningham | Margaret Park | 7 | 47.62 |
| Luis Rojas | Steve Johnson | 7 | 46.62 |
| Ladislav Kovács | Jane Peacock | 7 | 45.62 |
| Hugh O'Reilly | Jane Peacock | 7 | 45.62 |

| CustomerName | EmployeeName | invoices | total |
|---|---|---|---|

## Отчёт по продажам менеджеров (количество и сумма счетов)

Запрос извлекает данные о сотрудниках (формирует полное имя через объединение полей `FirstName` и `LastName` в `EmployeeName`), подсчитывает количество счетов-фактур (`Invoices`) и суммирует их общую стоимость (`Total`) для каждого сотрудника — с учётом клиентов, закреплённых за ним в качестве менеджера поддержки. Для этого выполняются соединения таблиц `Employee`, `Customer` (по полю `SupportRepId`) и `Invoice` (по `CustomerId`). Результаты группируются по идентификатору сотрудника (`e.EmployeeId`) и сортируются по убыванию суммарной стоимости счетов (`Total`). Функция `COALESCE` гарантирует замену возможных значений `NULL` на `0` в колонках `Invoices` и `Total`.

```
sql = """SELECT
    e.FirstName || ' ' || e.LastName AS EmployeeName,
    COALESCE(COUNT(i.InvoiceId), 0) AS Invoices,
    COALESCE(SUM(i.Total), 0) AS Total
FROM Employee e
LEFT JOIN Customer c ON c.SupportRepId = e.EmployeeId
LEFT JOIN Invoice i ON i.CustomerId  = c.CustomerId
GROUP BY e.EmployeeId
ORDER BY total DESC;"""
data = pd.read_sql_query(sql, conn)
display(Markdown(data.to_markdown(index=False)))
```

| EmployeeName | Invoices | Total |
|---|---|---|
| Jane Peacock | 146 | 833.04 |
| Margaret Park | 140 | 775.4 |
| Steve Johnson | 126 | 720.16 |
| Andrew Adams | 0 | 0 |
| Nancy Edwards | 0 | 0 |
| Michael Mitchell | 0 | 0 |
| Robert King | 0 | 0 |
| Laura Callahan | 0 | 0 |

## Ежемесячная статистика по продажам всех продуктов

Этот запрос извлекает агрегированную информацию о счетах (инвойсах) по месяцам: группирует записи по месяцам (`STRFTIME('%Y-%m', i.InvoiceDate)`), подсчитывает количество уникальных клиентов (`COUNT(c.CustomerId)`), общее количество счетов (`COUNT(i.InvoiceId)`) и суммарную стоимость счетов (`SUM(i.Total)`) для каждого месяца. Использует соединения: правое (`RIGHT JOIN`) между `Employee` и `Customer` по полю `SupportRepId`, левое (`LEFT JOIN`) между `Customer` и `Invoice` по `CustomerId`. Результаты сортируются по убыванию месяца (`InvoiceYear DESC`) и суммарной стоимости (`Total DESC`).

**Краткое название:** «Ежемесячная статистика по счетам» / «Monthly invoice stats»

```
sql = """SELECT
    STRFTIME('%Y-%m', i.InvoiceDate) AS InvoiceYear,
    COUNT(c.CustomerId) AS Customers,
    COUNT(i.InvoiceId) AS Invoices,
    SUM(i.Total) AS Total
FROM Employee e
RIGHT JOIN Customer c ON c.SupportRepId = e.EmployeeId
LEFT JOIN Invoice i ON i.CustomerId = c.CustomerId
```

```
GROUP BY STRFTIME('%Y-%m', i.InvoiceDate)
ORDER BY InvoiceYear DESC, Total DESC;
"""
data = pd.read_sql_query(sql, conn)
display(Markdown(data.head().to_markdown(index=False)))
```

| InvoiceYear | Customers | Invoices | Total |
|---|---|---|---|
| 2025-12 | 7 | 7 | 38.62 |
| 2025-11 | 7 | 7 | 49.62 |
| 2025-10 | 7 | 7 | 37.62 |
| 2025-09 | 7 | 7 | 37.62 |
| 2025-08 | 7 | 7 | 37.62 |

```
plt.figure(figsize=(12, 5))
plt.plot(data["InvoiceYear"], data["Total"], label="Продажи")
plt.plot(data["InvoiceYear"], data["Customers"], label="Клиенты")

plt.xticks(rotation="vertical")
plt.grid(True, alpha=0.3)
plt.xlabel("Год")
plt.ylabel("Продажи / Клиенты")
plt.title("Продажи / Кол-во клиентов по годам")
plt.legend()

plt.show()
```

Figure 3: png

## Альбомы с наибольшей суммой продаж треков

**Описание запроса:**

Запрос извлекает название альбома (`Title`), подсчитывает количество треков в каждом альбоме (`track_count`) и вычисляет общую сумму продаж по всем заказам

для треков этого альбома (ammount). Для этого он объединяет таблицы InvoiceLine, Track и Album через LEFT JOIN, группирует результаты по идентификатору альбома (AlbumId) и сортирует итоговую таблицу по сумме продаж в порядке убывания (от наибольшего к наименьшему).

```
sql = """SELECT
    a.Title,
    COUNT(il.TrackId ) as track_count,
    SUM(il.UnitPrice * il.Quantity) as ammount
FROM InvoiceLine il
LEFT JOIN Track t ON il.TrackId = t.TrackId
LEFT JOIN Album a ON t.AlbumId  = a.AlbumId
GROUP BY a.AlbumId
ORDER BY ammount DESC;
"""
data = pd.read_sql_query(sql, conn)
display(Markdown(data.head().to_markdown(index=False)))
```

| Title | track_count | ammount |
|---|---|---|
| Battlestar Galactica (Classic), Season 1 | 18 | 35.82 |
| The Office, Season 3 | 16 | 31.84 |
| Minha Historia | 27 | 26.73 |
| Lost, Season 2 | 13 | 25.87 |
| Heroes, Season 1 | 13 | 25.87 |

## Самые продаваемые альбомы: название, число проданных треков, выручка и её доля от общего объёма

**Описание в одном абзаце:**

Запрос формирует отчёт по продажам музыкальных альбомов: для каждого альбома выводит его название (a.Title), количество проданных треков (track_count), суммарную выручку от их продаж (ammount), а также процентную долю этой выручки от общей суммы продаж всех альбомов (persent, округлённую до двух знаков после запятой). Данные собираются через соединение таблиц InvoiceLine, Track и Album, группируются по идентификатору альбома (a.AlbumId), а итоговая выборка сортируется по убыванию процентной доли выручки.

```
sql = """SELECT
    a.Title,
    COUNT(il.TrackId ) as track_count,
    SUM(il.UnitPrice * il.Quantity) as ammount,
    ROUND((SUM(il.UnitPrice * il.Quantity) * 100) /
        SUM(SUM(il.UnitPrice * il.Quantity)) OVER (), 2) as percent
FROM InvoiceLine il
LEFT JOIN Track t ON il.TrackId = t.TrackId
LEFT JOIN Album a ON t.AlbumId  = a.AlbumId
GROUP BY a.AlbumId
ORDER BY percent DESC;
"""
data = pd.read_sql_query(sql, conn)
display(Markdown(data.head().to_markdown(index=False)))
```

| Title | track_count | ammount | percent |
|---|---|---|---|
| Battlestar Galactica (Classic), Season 1 | 18 | 35.82 | 1.54 |
| The Office, Season 3 | 16 | 31.84 | 1.37 |
| Minha Historia | 27 | 26.73 | 1.15 |
| Greatest Hits | 26 | 25.74 | 1.11 |
| Heroes, Season 1 | 13 | 25.87 | 1.11 |